Project Phase 2 Deliverable 2: Proof of Concept Implementation

Shreya Rai

University of the Cumberlands

Course Number: MSCS-532-B01

Michael Solomon

March 29, 2026

Project Phase 2 Deliverable 2: Proof of Concept Implementation

Abstract

The proof-of-concept (PoC) implementation of a Phase 1 Python-based e-commerce recommendation system is shown in this article. This phase's main goal is to build the fundamental elements of the chosen data structures and show how they work in a safe setting. To effectively handle user-product interactions and produce suggestions, the system makes use of hash tables, graphs, heaps, and sparse matrix representations. Important functions including insertion, retrieval, and ranking are the main focus of the implementation. Test cases are used to verify accuracy, and development-related difficulties are talked about. Lastly, potential improvements needed for widespread implementation are described.

1. Partial Implementation

Implementing the fundamental data structures revealed in Phase 1 is the main goal of the proof-of-concept. These structures facilitate effective data processing, retrieval, and storage and serve as the foundation of the recommendation engine.

1.1. User Data Hash Tables

To store user interactions with products, a dictionary-based framework was put in place. Constant-time operations for insertion and retrieval are made possible by mapping each user to a dictionary of product ratings. In order to swiftly access user preferences during recommendation generation, this structure is essential.

1.2. Relationship Graph Representation

The relationships between users and products were modeled using an adjacency list graph structure. Traversal-based recommendation methods, including collaborative filtering, are made possible by each user node connecting to products they have engaged with.

1.3. Recommendation Engine Based on Heaps

To effectively maintain the top-N suggested products, a priority queue (min-heap) was put in place. The heap minimizes computational cost by ensuring that only the most pertinent products are kept rather than sorting every product.

1.4. Representation of Sparse Matrices

A dictionary-based sparse matrix representation was employed to address the sparsity of the data. Only non-zero user-product interactions are stored, significantly reducing memory usage and improving scalability.

Together, these elements create a modular design in which every data structure has a distinct function within the recommendation pipeline.

2. Demonstration and Testing

Test cases were created to mimic user interactions and assess the accuracy of crucial processes in order to validate the implementation.

2.1. Testing for functionality

Sample user-product interactions were used to test the system. The graph was updated with new users, ratings, and relationships. The heap-based function successfully returned the top-N products after generating a set of recommendation ratings.

2.2. A Sample Test Case

Three goods with different scores participated in a sample test. The efficiency of the heap-based method was demonstrated by the system's accurate identification of the two highest-scoring products.

2.3. *Testing Edge Cases*

A number of edge cases were assessed:

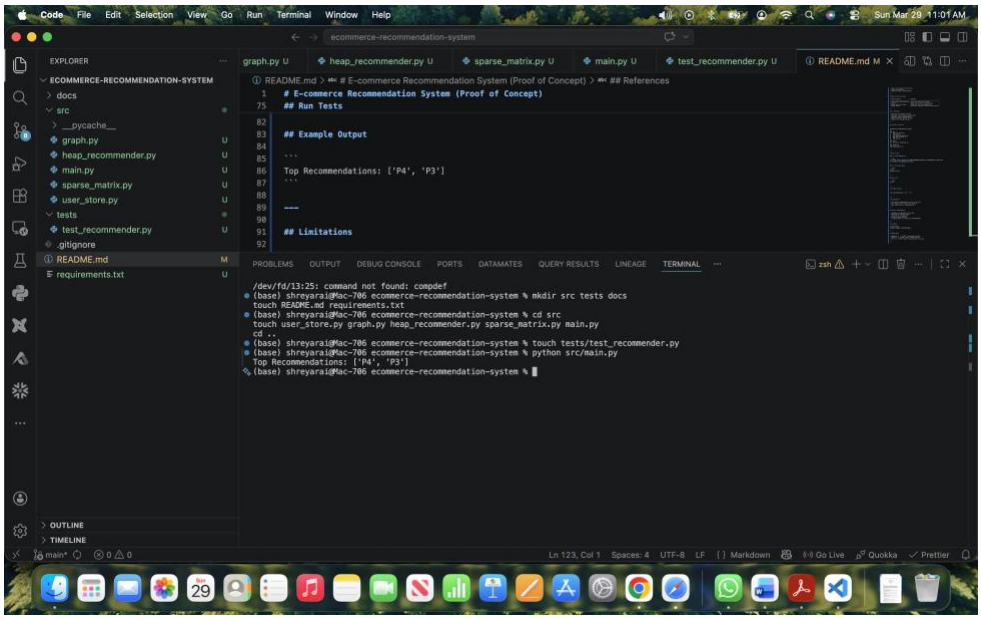
- There were no suggestions from empty user data.
- The hash database was appropriately updated with duplicate ratings.
- Performance deterioration was avoided when handling large N values.

2.4. *Test Outcomes*

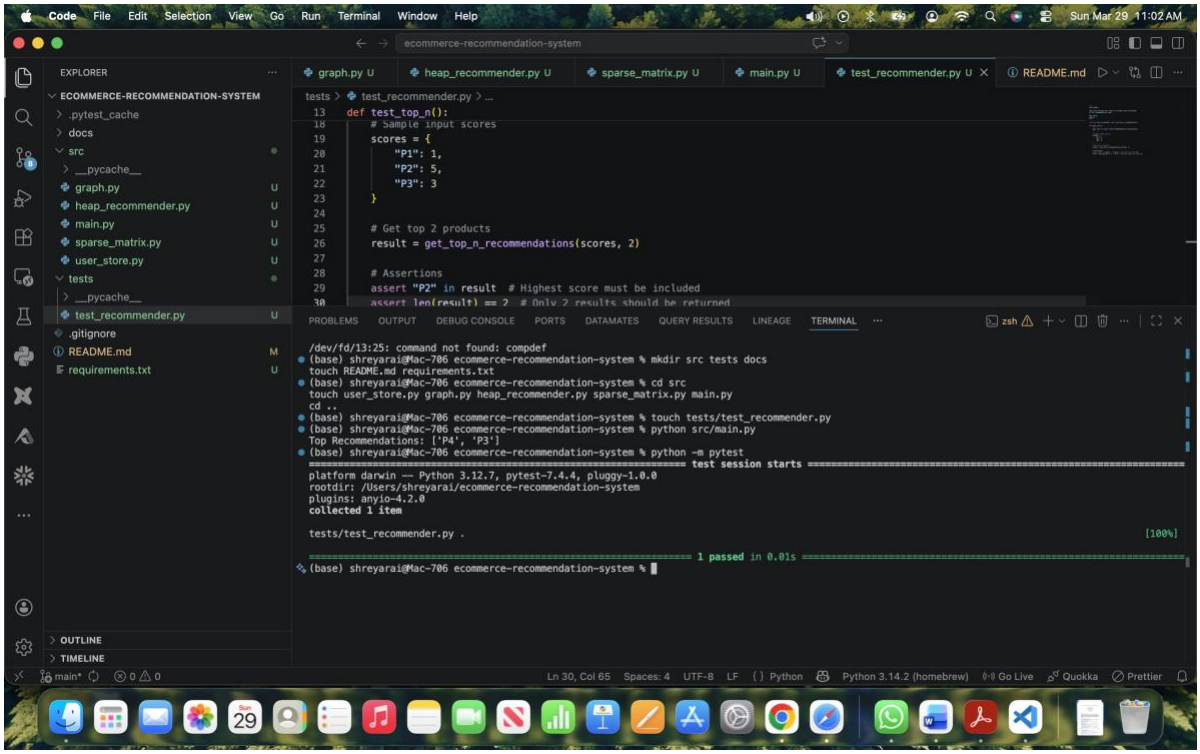
The findings verified that:

- Data retrieval and insertion processes work properly.
- The proper ranking order is maintained by the heap structure.
- In both fundamental and edge-case circumstances, the system performs as anticipated.

ScreenShot1: Output of Top-N Recommendation Function:



ScreenShot2: The system correctly returns the top recommendations.



3. Implementation Challenges and Solutions

3.1. Synchronization of Data Between Structures

Challenge: Maintaining consistency across sparse matrices, graphs, and hash tables.

Solution: Standardized user-product identifiers for all data structures.

3.2. Managing Sparse Data:

Challenge: The majority of users deal with a small number of items.

Solution: To store only significant interactions, a sparse matrix was implemented using dictionaries.

3.3. Effective Generation of Recommendations

Challenge: Sorting big datasets requires a lot of processing power.

Solution: The complexity was reduced from O(n log n) to O(n log k) by using heap-based priority queues.

3.4. Code Modularity

Challenge: Preserving adaptability for upcoming improvements.

Solution: To guarantee scalability and maintainability, distinct modules were created for every data structure.

4. *Next Steps*

The following enhancements are necessary to turn the PoC into a system that is ready for production:

- Use similarity metrics like Pearson correlation and cosine similarity.
- Incorporate machine learning methods such as matrix factorization.
- Use optimized libraries (like SciPy) in favor of simple sparse representation.
- Improve response times by implementing caching techniques.
- Expand the system to accommodate real-time user interactions and big databases.
- Carry out performance optimization and benchmarking.

5. Code Snippets and Documentation

For every data structure, the implementation is divided into distinct Python files and is modular. Important elements consist of:

- User data management with the UserStore class
- Graph class for relationship modeling
- Top-N suggestions using a heap-based function

- Implementation of sparse matrices for memory efficiency

You may access the whole implementation at: https://github.com/ShreyaRai20/ecommerce-recommendation-system.git

Every single module includes detailed comments and documentation to ensure readability and maintainability.

6. Conclusion

A functional proof-of-concept for an e-commerce recommendation system is successfully demonstrated in this phase. The implementation demonstrates the value of effective data structures in solving practical issues and confirms the viability of the design suggested in Phase 1. Even though the system only functions on a limited scale now, it lays a solid basis for future optimization and scalability enhancements.

References

Aggarwal, C. C. (2016). Recommender systems: The textbook. Springer.

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30–37. https://doi.org/10.1109/MC.2009.263

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. Proceedings of the 26th International Conference on World Wide Web, 173–182. https://doi.org/10.1145/3038912.3052569